

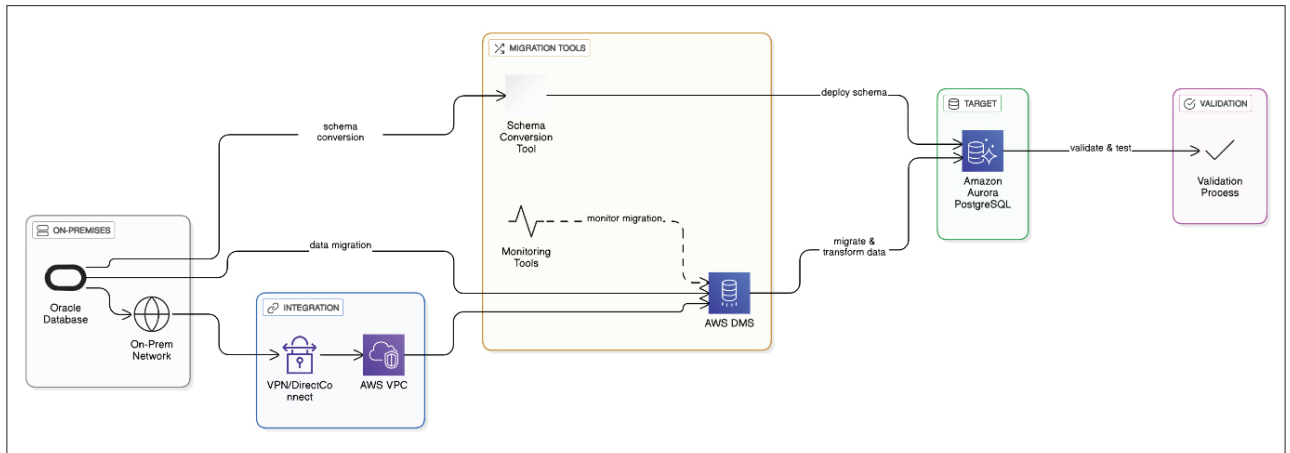


Figure 2: AWS reference architecture for heterogeneous data migration

Imagine a financial institution operating a legacy Oracle database on-premises, seeking to modernise by migrating to Amazon Aurora PostgreSQL on AWS. This transition is not straightforward as Oracle and PostgreSQL have distinct ways of handling data, constraints, and stored procedures. Therefore, AWS DMS works in tandem with the AWS Schema Conversion Tool (SCT), which analyses the source Oracle schema and generates conversion scripts for Aurora PostgreSQL.

This process requires careful mapping of data types and business logic to ensure that the target database functions as intended.

Pain areas in AWS database migration

While cloud adoption is a strategic imperative, the actual mechanism of moving data—specifically heterogeneous migration—remains one of the most daunting tasks for IT leaders. The manuscript touches upon ‘complex schemas’ and ‘downtime’, but the pain points run significantly deeper when analysed through a technical lens.

The primary bottleneck in database migration is arguably the schema conversion ‘impedance mismatch’. When migrating from legacy commercial engines (like Oracle or SQL Server) to open source cloud-native engines (like Amazon Aurora PostgreSQL), the challenge is rarely the data itself—it is the application logic residing in the database. Proprietary procedural code (PL/SQL or T-SQL) often contains decades of business logic, bespoke functions, and proprietary extensions that have no direct equivalent in the target database. Traditional rule-based tools, such as the standard AWS Schema Conversion Tool (SCT), are excellent at converting about 80% of this code automatically. However, the remaining 20%—often referred to as the ‘last mile’—requires manual rewriting. This manual intervention is error-prone, requires niche expertise, and often delays project timelines by months.

Secondly, performance degradation risks pose a significant psychological and operational barrier. A database tuned for twenty years on-premises has execution plans optimised for specific hardware and storage subsystems. Moving this to a cloud architecture can initially result in query regression, where queries that ran in milliseconds suddenly take seconds. Identifying these regressions before the cutover is difficult because simulating full production loads in a staging environment is resource-intensive and complex.

Finally, data integrity and validation at scale is a massive pain point. In multi-terabyte migrations, verifying that every row, constraint, and index has been replicated accurately is computationally expensive. Traditional checksum validations can slow down the migration process itself. The fear of ‘silent data corruption’—where data types are coerced incorrectly during transit without throwing an error—keeps database administrators up at night. This anxiety often leads to extended ‘parallel run’ periods where both old and new systems are kept active, doubling costs and operational overhead.

Using generative AI in database migration

The integration of generative AI into the database migration lifecycle marks a shift from ‘deterministic translation’ to ‘semantic understanding’. Historically, migration tools operated like a dictionary: if they saw Word A in the source, they replaced it with Word B in the target. If Word A had no direct translation, the tool failed. Generative AI, specifically large language models (LLMs) integrated into services like AWS DMS Schema Conversion, changes this paradigm by analysing the ‘intent’ of the code.

Using generative AI in this context involves a multi-step workflow that mimics a senior database engineer’s reasoning. When the migration service encounters a